Servia



Course: DBS(Database Systems)

Section: 5A

Group Members:

Safia Mussarat [23k-0716]

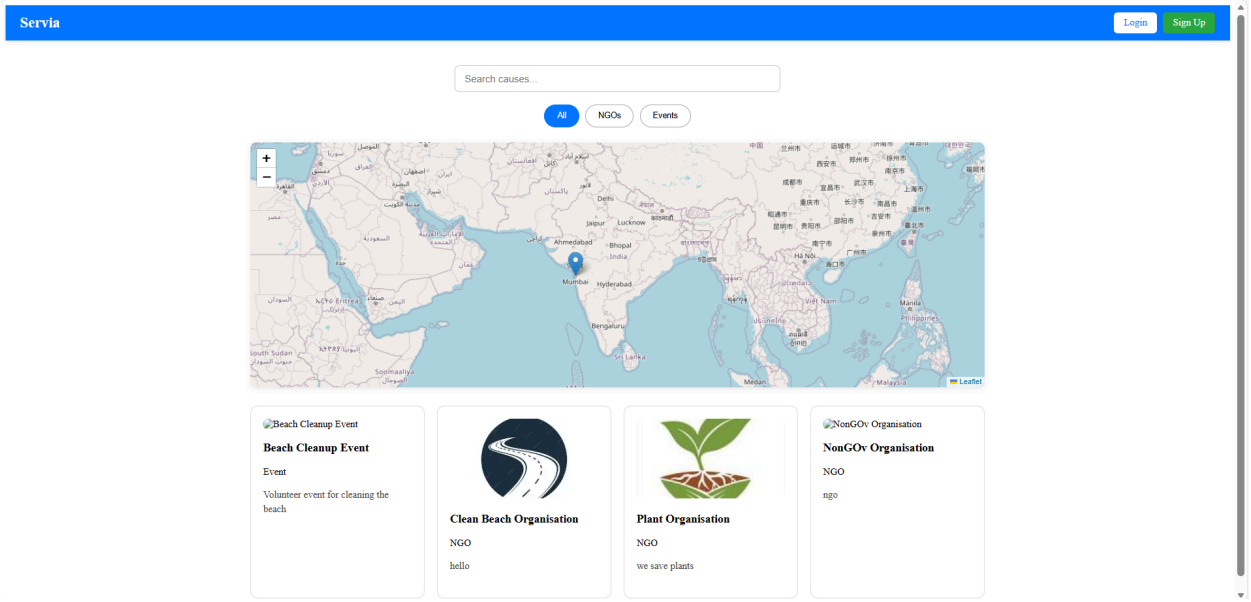
Meghna [23k-0507]

Fatimah Riaz [23k-0687]

Project Overview

Servia is a centralized online platform designed to showcase NGOs and foundations operating within a city. It provides admins and NGOs the ability to manage information while enabling visitors to browse through a searchable and filterable gallery and map of NGOs and causes. Signed up and verified users can The goal is to improve transparency, user engagement, and streamline NGO management at a community level.

HomePage:



Targeted Audience(Safe for all):

- Admin: Responsible for managing users, causes, and content ensuring platform integrity and data accuracy.
- User: Can manage user details including feedback, donations, account details and volunteer opportunities and related NGO's.
- Cause: Can manage cause details including events, locations, donations, account details and volunteer opportunities.
- Visitor: Can browse causes and NGOs, explore events.

The platform supports secure login with password hashing, ensuring safe authentication for all users.

Project Scope:

Platform Functionality

- Centralized directory for city-wide NGOs and causes.
- Interactive homepage with filtering and map-based browsing.
- Secure login system with hashed passwords and JWT authentication.
- Admin dashboards for data monitoring and verification.
- User dashboards for data manipulation.
- User engagement features such as donations, volunteer registration, and feedback.
- NGO ability to create events with geolocation integration through Leaflet maps.

Technology Scope

- **Frontend:** React, CSS, Leaflet
- **Backend:** Flask, Flask-SQLAlchemy, Flask-Login, JWT, Flask-CORS
- **Database:** PostgreSQL / SQLite
- **Environment:** python-dotenv for configuration

Data Scope

- Storage of causes, events, NGOs, users, social links, rating feedback, and donation/volunteer records.
- Map location information for visualization.
- Authentication and role management through AuthData.

Functionalities

Functional Requirements

1 User & Authentication

- Users must be able to register with hashed passwords.
- Login and logout functionality must be secure using JWT.
- Admins must be able to verify users and NGOs.
- Authenticated users can interact with the platform (donate, volunteer, give feedback).

2 Admin Features

- Admin can:
    - View all users and causes.
    - Verify/unverify users and causes.
    - Delete users, NGOs, and associated data.
    - Access protected admin-only routes.

3 Cause & NGO Management

- Users can:
    - Create and manage causes.
    - Upload logos, descriptions, and social links.
    - Create events with date, time, capacity.
    - Manage cause locations (city, coordinates, contact info).

4 Interaction Features

- Users can:
    - Donate to causes.
    - Volunteer for causes.
    - Submit feedback (ratings and comments).
    - Check their info

- Visitors can:
    - Browse causes, NGOs, events.
    - View locations on a map.

5 Data Retrieval

- Users and visitors must be able to search and filter causes.
- API must return:
    - Cause details
    - Related events, feedback, locations, social links, account details
    - NGO details linked to a cause

6 System Operations

- System must automatically create a default admin if none exists.
- All admin routes must be protected via decorators.

7 Cli.py

- To show transactions and triggers, because they were not used for the web development portion.

Non-Functional Requirements

1 Security

- Passwords must be hashed before storage.
- JWT must be used for secure session management.

- Admin operations must be role-restricted.
- Sensitive operations must use authentication tokens.

2 Performance

- Search and filtering should return results quickly.
- Map rendering should be efficient with Leaflet.

3 Reliability

- Database migrations must be handled using Flask-Migrate.
- API must prevent unauthorized data manipulation.

4 Usability

- UI must be clear, responsive, and accessible.
- Navigation between NGOs, causes, and events must be intuitive.

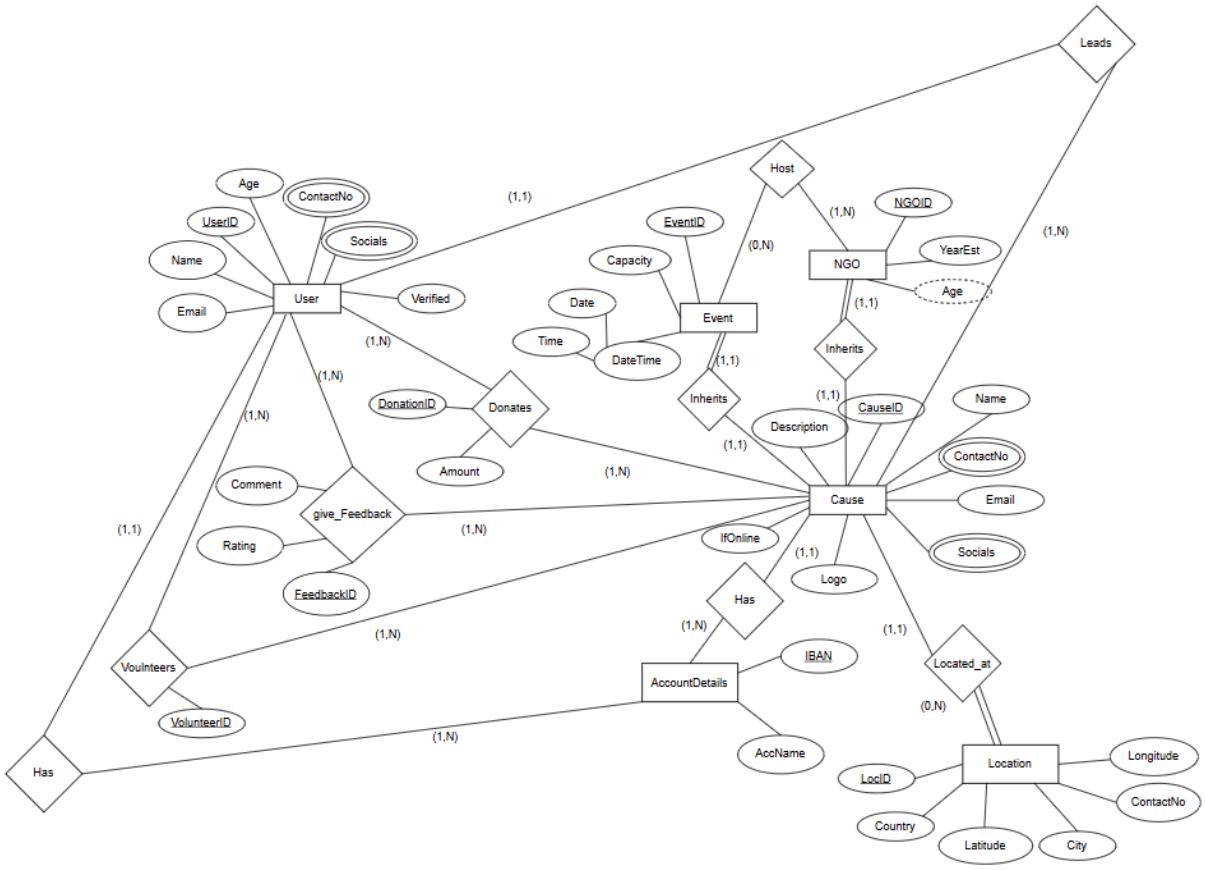
5 Scalability

- System should allow easy addition of more causes, NGOs, and users.
- Backend and database must scale for increased requests.

6 Maintainability

- Clear separation of frontend and backend code.
- Modular route and model structure in Flask.
- Environment variables must control configuration.

ER Diagram:



UnNormalized ERD:
https://app.diagrams.net/#G1ss7at_vk0Dmmtz7lK4ZQIhgNHiYUi96E#%7B%22pageId%22%3A%22KmnN7jU-c_MA_Xl4-vz9%22%7D

Relational Schema(Normalized):

- Cause(CauseID[PK], Email, IfOnline, Logo, Name, Description, UserID[FK])
- User(UserID[PK], Age, Verified, Name, Email)
- Event(EventID[PK], Capacity, Date,Time, CauseID[FK], NGOID[FK])
- NGO(NDOID[PK],YearEst,Age,CauseID[FK])
- Location(LocID[PK], Country, Latitude, Longitude, City, ContactNo, CauseID[FK])
- AccountDetails(IBAN[PK], AccName, UserID[FK], CauseID[FK])
- Feedback(FeedbackID[PK], Comment, Rating, CauseID[FK], UserID[FK]) —-Associative Relation Table
- Donation(DonationID[PK], Amount, UserID[PK], CauseID[FK]) —-Associative Relation Table
- Volunteer(VoluteerID[PK], UserID[PK], CauseID[FK]) —-Associative Relation Table

- UserContact(ContactID[PK], Contact, UserID[FK])
- UserSocials(SocialID[PK], Social, UserID[FK])
- CauseContact(ContactID[PK], Contact, CauseID[FK])
- CauseSocials(SocialID[PK], Social, CauseID[FK])
- auth_data(id,name,role,verified, fk_id) -- added during production for login functionality

Key models include:

- AuthData: Manages authentication and roles (user, NGO, event).
- User: Stores user details, roles, and relationships to causes, donations, feedback, and volunteering.
- Cause: Central entity for causes with linked NGOs, events, locations, donations, and social contacts.
- NGO: Associated with a cause, stores NGO-specific details.
- Event and Location: Handling event dates, capacities, and geolocation.
- Supporting tables for feedback, donations, volunteers, and contact details ensure rich interaction data.

Conclusion:

Servia serves as a modern, secure, and efficient platform for managing and showcasing NGOs and community causes. By integrating interactive features such as donations, volunteering, event management, and map-based views, it enhances community engagement and transparency. The system's robust role management and admin verification ensure data integrity, while its scalable architecture and React-Flask ecosystem provide a strong foundation for future growth.